COMP3320/6464 Week 3, Lecture 3

Rhys Hawkins

Semester 2, 2024

Previously

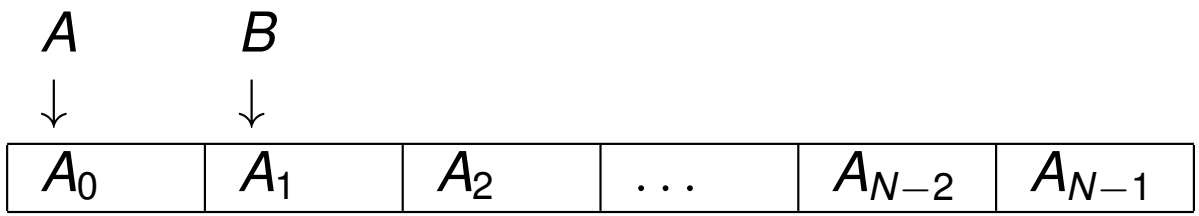
Performance improvement due to

- Pipelining
- Loop unrolling
- Breaking loop dependencies

Pointer aliasing

- C is a low level language
- Pointers are unrestricted can point anywhere, even to illegal addresses
- Pointers to arrays can be aliased

```
void saxpy(int N, float s, float *x, float *y)
{
  int i;
  for (i = 0; i < N; i ++) {
    y[i] = s*x[i] + y[i];
  }
}
float A[N];
float *B = &(A[1]);
saxpy(N - 1, 1.0, A, B);
```



Pointer Aliasing

- What happens if we loop unroll?

```
void saxpy_x2(int N, float s, float *x, float *y)
{
  int i;
  for (i = 0; i < N; i += 2) {
    float x1 = s * x[i];
    float x2 = s * x[i + 1];

    y[i] = y[i] + x1;
    y[i + 1] = y[i + 1] + x2;
  }
}
```

Original

1. `A[1] = A[1] + s*A[0]`
2. `A[2] = A[2] + s*A[1]`

Aliasing causes hidden dependency between iteration 1 and 2 for `A[1]`

Unrolled

1. `x1 = s*A[0]; x2 = s*A[1]; A[1] = A[1] + x1`
2. `A[2] = A[2] + x2`

Local variable breaks dependency and gives incorrect result (uses previous value)

Pointer Aliasing : `restrict`

- The `restrict` keyword can be used to require distinct pointers for functions, eg:

```
void saxpy_x2(int N, float s, float *restrict x, float *restrict y);
```

This achieves a number of things:

- Tells users of the function that you shouldn't call this function with aliased pointers
- Allows the compiler (sometimes) to give warnings when called with aliased pointers
- Allows the compiler to assume the pointers are unaliased leading to improved optimisation

Good practice to use the restrict keyword when using loop unrolling

Analyzing Loop Performance

Consider a dot product code

```
int i;
float s = 0.0;
float A[N], B[N];
for (i = 0; i < N; i ++) {
  s = s + A[i] * B[i];
}
```

- Use latencies from Gadi: 3 cycle Load/Store, 4 cycle Fadd/Fmul
- Assumes required memory is already in L1 Cache
- Assume loop is broken into micro-ops: Load A_i, Load B_i, Fmul $A_i \times B_i$, Fadd s = s + $A_i \times B_i$
- How many cycles does each iteration take (neglecting loop branches)

Analyzing Loop Performance : Non-pipelined

First consider a non-pipelined CPU

Cycle	Op
1	Load A_0
2	
3	
4	Load B_0
5	
6	
7	Fmul $A_0 \times B_0$
8	
9	
10	
11	Fadd s = s + $A_0 \times B_0$
12	
13	
14	
15	Load A_1
⋮	

- 14 Cycles for 2 floating point operations

Analyzing Loop Performance : Pipelined

Assume a basic pipelined CPU with out-of-order execution

Cycle	Op
1	Load A_0
2	Load B_0
3	
4	
5	Fmul $A_0 \times B_0$
6	
7	
8	
9	Fadd s = s + $A_0 \times B_0$
⋮	

- The main consequence of pipelining is that we don't have to wait for independent instructions
- Loads can be executed immediately after each other

Analyzing Loop Performance : Pipelined

Assume a basic pipelined CPU with out-of-order execution

Cycle	Op
1	Load A_0
2	Load B_0
3	Load A_1
4	Load B_1
5	Fmul $A_0 \times B_0$
6	
7	Fmul $A_1 \times B_1$
8	
9	Fadd s = s + $A_0 \times B_0$
10	
11	
12	
13	Fadd s = s + $A_1 \times B_1$
⋮	

- Loads for the next iteration (red) can be interleaved with operations from previous iteration
- Why is red Fadd at 13 rather than 11? What breaks the pattern?

Analyzing Loop Performance : Pipelined

Assume a basic pipelined CPU with out-of-order execution

Cycle	Op
1	Load A_0
2	Load B_0
3	Load A_1
4	Load B_1
5	Fmul $A_0 \times B_0$
6	Load A_2
7	Fmul $A_1 \times B_1$
8	Load B_2
9	Fadd s = s + $A_0 \times B_0$
10	Load A_3
11	Fmul $A_2 \times B_2$ *
12	Load B_3
13	Fadd s = s + $A_1 \times B_1$
14	Load A_4
15	Fmul $A_3 \times B_3$
16	Load B_4
17	Fadd s = s + $A_2 \times B_2$
⋮	

Analyzing Loop Performance : Pipelined

Steady state operation of a basic pipelined CPU with out-of-order execution

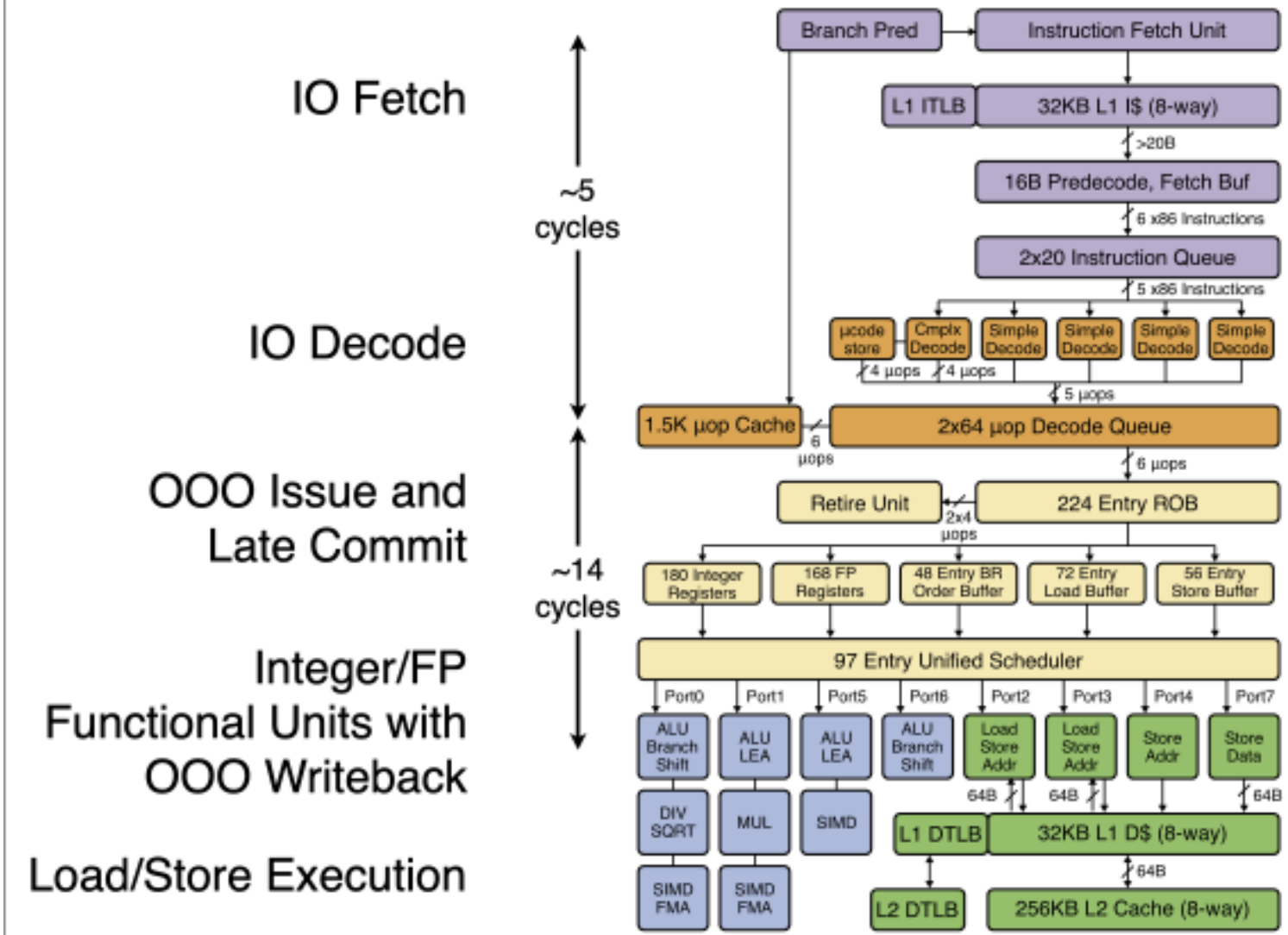
Cycle	Op
1	Fadd s = s + $A_i \times B_i$
2	Load A_{i+3}
3	Fmul $A_{i+2} \times B_{i+2}$
4	Load B_{i+3}
5	Fadd s = s + $A_{i+1} \times B_{i+1}$
⋮	

- 4 Cycles for 2 Floating point operations
- Out of order execution ensures an instruction is started every cycle

Superscalar Architecture

- A single pipeline is able to start a new (indepedent) micro-operation each cycle
- In a superscalar CPU, we have multiple (sometimes specialised) pipelines that can each start a new (independent) micro-operation each cycle
- Most modern Intel CPUs can do 2 loads, 2 floating point operations and a store each cycle
- Allows Instruction Level Parallelism.

Intel Skylake Architecture



Analyzing Loop Performance: Superscalar

Assume a super-scalar CPU with out-of-order execution with 2 load, 2 floating point operations per cycle (dot product doesn't need store)

Cycle	Load	Load	FP Op	FP Op
1	Load A_0	Load B_0		
2				
3				
4			Fmul $A_0 \times B_0$	
5				
6				
7				
8				Fadd s = s + $A_0 \times B_0$

Analyzing Loop Performance: Superscalar

Assume a super-scalar CPU with out-of-order execution with 2 load, 2 floating point operations per cycle (dot product doesn't need store)

Cycle	Load	Load	FP Op	FP Op
1	Load A_0	Load B_0		
2	Load A_1	Load B_1		
3				
4			Fmul $A_0 \times B_0$	
5			Fmul $A_1 \times B_1$	
6				
7				
8				Fadd s = s + $A_0 \times B_0$
9				
10				
11				
12				Fadd s = s + $A_1 \times B_1$

Analyzing Loop Performance: Superscalar

Assume a super-scalar CPU with out-of-order execution with 2 load, 2 floating point operations per cycle (dot product doesn't need store)

Cycle	Load	Load	FP Op	FP Op
1	Load A_0	Load B_0		
2	Load A_1	Load B_1		
3	Load A_2	Load B_2		
4			Fmul $A_0 \times B_0$	
5			Fmul $A_1 \times B_1$	
6			Fmul $A_2 \times B_2$	
7				
8				Fadd s = s + $A_0 \times B_0$
9				
10				
11				
12				Fadd s = s + $A_1 \times B_1$
13				
14				
15				
16				Fadd s = s + $A_2 \times B_2$

- Loop carried dependency on *s* prevents optimal use of superscalar architecture

Analyzing Loop Performance: Superscalar

Steady state

Cycle	Load	Load	FP Op	FP Op
1	Load A_{i+1}	Load B_{i+1}	Fmul $A_i \times B_i$	Fadd s = s + $A_{i-1} \times B_{i-1}$
2				
3				
4				
5	Load A_{i+2}	Load B_{i+2}	Fmul $A_{i+1} \times B_{i+1}$	Fadd s = s + $A_i \times B_i$

- 4 Cycles for 2 Floating point operations. Same as for a single pipeline!
- Loop carried dependency is preventing instruction level parallelism.

Loop unroll with Auxiliary Accumulators

- Use x2 loop unroll with two summation variables
- s1 and s2 additions are independent
- Assume N is divisible by 2/Ignore extra loop elements for now

```
int i;
float s;
float s1 = 0.0, s2 = 0.0;
float A[N], B[N];
for (i = 0; i < N; i += 2) {
  s1 = s1 + A[i] * B[i];
  s2 = s2 + A[i + 1] * B[i + 1]
}
s = s1 + s2;
```

Analyzing Loop Performance: Superscalar with x2 Loop unrolling

Assume a super-scalar CPU with out-of-order execution with 2 load, 2 floating point operations per cycle (dot product doesn't need store)

Cycle	Load	Load	FP Op	FP Op
1	Load A_0	Load B_0		
2	Load A_1	Load B_1		
3				
4			Fmul $A_0 \times B_0$	
5			Fmul $A_1 \times B_1$	
6				
7				
8				Fadd s1 = s1 + $A_0 \times B_0$
9				Fadd s2 = s2 + $A_1 \times B_1$

- Looks good
- What about the next iteration

Analyzing Loop Performance: Superscalar with x2 Loop unrolling

Assume a super-scalar CPU with out-of-order execution with 2 load, 2 floating point operations per cycle (dot product doesn't need store)

Cycle	Load	Load	FP Op	FP Op
1	Load A_0	Load B_0		
2	Load A_1	Load B_1		
3	Load A_2	Load B_2		
4			Fmul $A_0 \times B_0$	
5			Fmul $A_1 \times B_1$	
6			Fmul $A_2 \times B_2$	
7				
8				Fadd s1 = s1 + $A_0 \times B_0$
9				Fadd s2 = s2 + $A_1 \times B_1$
10				
11				
12				Fadd s1 = s1 + $A_2 \times B_2$

- Still have a dependency of s1 and s2 that leaves no-ops in pipes

Analyzing Loop Performance: Superscalar with x2 Loop unrolling

Steady State

Cycle	Load	Load	FP Op	FP Op
1	Load A_{i+1}	Load B_{i+1}		
2			Fmul $A_i \times B_i$	Fadd s1 = s1 + $A_{i-2} \times B_{i-2}$
3	Load A_{i+2}	Load B_{i+2}		
4			Fmul $A_{i+1} \times B_{i+1}$	Fadd s2 = s2 + $A_{i-1} \times B_{i-1}$

- 4 Cycles for 4 Floating point operations
- Effectively twice the performance
- What about unrolling again?

Loop unroll with Auxiliary Accumulators

- Use x4 loop unroll with two summation variables
- s1, s2, s3 and s4 additions are independent
- Assume N is divisible by 4/Ignore extra loop elements for now

```
int i;
float s;
float s1 = 0.0, s2 = 0.0, s3 = 0.0, s4 = 0.0;
float A[N], B[N];
for (i = 0; i < N; i += 2) {
  s1 = s1 + A[i] * B[i];
  s2 = s2 + A[i + 1] * B[i + 1]
  s3 = s3 + A[i + 2] * B[i + 2]
  s4 = s4 + A[i + 3] * B[i + 3]
}
s = s1 + s2 + s3 + s4;
```

Analyzing Loop Performance: Superscalar with x2 Loop unrolling

Assume a super-scalar CPU with out-of-order execution with 2 load, 2 floating point operations per cycle (dot product doesn't need store)

Cycle	Load	Load	FP Op	FP Op
1	Load A_0	Load B_0		
2	Load A_1	Load B_1		
3	Load A_2	Load B_2		
4	Load A_3	Load B_3	Fmul $A_0 \times B_0$	
5	Load A_4	Load B_4	Fmul $A_1 \times B_1$	
6			Fmul $A_2 \times B_2$	
7			Fmul $A_3 \times B_3$	
8			Fmul $A_4 \times B_4$	Fadd s1 = s1 + $A_0 \times B_0$
9				Fadd s2 = s2 + $A_1 \times B_1$
10				Fadd s3 = s3 + $A_2 \times B_2$
11				Fadd s4 = s4 + $A_3 \times B_3$
12				Fadd s1 = s1 + $A_4 \times B_4$

- No more stalls
- Full instruction level parallelism for this loop achieved.
- Loop Unrolling x8 gives marginal improvement again (but not the same doubling).

Analyzing Loop Performance: Superscalar with x4 Loop unrolling

Steady State

Cycle	Load	Load	FP Op	FP Op
1	Load A_{i+3}	Load B_{i+3}	Fmul $A_i \times B_i$	Fadd s1 + $A_{i-4} \times B_{i-4}$
1	Load A_{i+4}	Load B_{i+4}	Fmul $A_{i+1} \times B_{i+1}$	Fadd s2 + $A_{i-3} \times B_{i-3}$
1	Load A_{i+5}	Load B_{i+5}	Fmul $A_{i+2} \times B_{i+2}$	Fadd s3 + $A_{i-2} \times B_{i-2}$
1	Load A_{i+6}	Load B_{i+6}	Fmul $A_{i+3} \times B_{i+3}$	Fadd s4 + $A_{i-1} \times B_{i-1}$

- 4 Cycles for 8 Floating point operations

Verification on GADI

- We can use PAPI to count both the number of cycles and number of floating point operations
- Tested using a repeat loop resulting in 320 million FLOPs

	Cycles	FLOPs/cycle	MFLOPs
Original	640,268,139	0.500	1800
Loop Unroll x2	320,135,565	1.000	3200
Loop Unroll x4	160,066,268	1.999	6300

- Performance matches expectations
- Factor of 4 improvement in speed

Loop Analysis

Key things to keep in mind:

- Superscalar CPUs with out-of-order execution
  - Simple analysis shown here can help identify improvements in your own code.
- Loop unrolling is generally only needed when loop dependencies exist that a compiler can't optimise (C standards/Floating point Rules)
- Break loop dependencies to allow greater Instruction Level Parallelism
- These analyses assume that required memory is in L1 Cache
- In general unrolling is inadvisable when loop
  - has a low number of iterations
  - is large/complex
  - has dynamic termination conditions
  - calls external functions

Next week

- Memory access optimisations (read and write)
- Mid-Semester Test (Tue 13th August)